

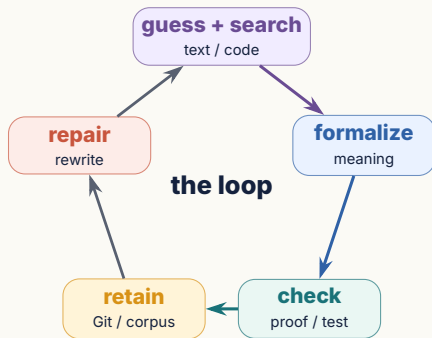
Is the QED Utopia Upon Us?

From Autoformalization to
Machine-Understandable
Science, Software and Beyond

Josef Urban

AI4REASON · University of Gothenburg

AGI 2026



AGI'18: the semantic AI paradise

JOKE

CLAIM

**"No one shall drive us from the
semantic AI paradise of
computer-understandable math and
science."**

At the time this was partly a slogan.
Now it is increasingly a workflow.

J. Urban, AGI 2018. This talk revisits the slogan with 2026 evidence.

My 2026 Update

OBSERVED

Production changed.

Large mathematical texts can now be turned into checked formal artifacts much faster than expected.

CLAIM

The loop matters. The interesting object is not one model, but the loop between guessing, search, checking, repair, libraries and training data.

CONJECTURE

This may generalize.

Similar loops may reshape software, scientific modeling and some structured public reasoning.

Caveat. This is not a claim that all mathematics is solved, or that current agents understand everything, or that society should be run by a theorem prover.

The QED Manifesto (Utopia) – 1994

The development of mathematics toward greater precision has led, as is well known, to the formalization of large tracts of it, so that one can prove any theorem using nothing but a few mechanical rules.

– K. Godel

QED is the very tentative title of a project to **build a computer system that effectively represents all important mathematical knowledge** and techniques.

– QED Manifesto

The idea is to make a language such that everything we write in it is interpretable as correct mathematics ... This may include the writing of a vast mathematical encyclopedia, to which everybody (either a human or a machine) may contribute what he likes. The idea of a kind of formalized encyclopedia was already conceived and partly carried out by Peano around 1900, but that was still far from what we might call automatically readable.

– N. G. de Bruijn

Mike Beeson, 2014: what is holding up QED?

The QED singularity: the future time when formal proofs become normal in mathematics.

Beeson listed the painful blockers:

- too many steps in too much detail;
- systems not knowing ordinary undergraduate facts;
- unnatural proof writing compared with $\text{T}_\text{E}\text{X}$;
- the wrong mixture of logic and computation.

Takeaway. The 2026 surprise is that the cost term in Beeson-style cost-benefit analysis seems to be changing very fast.

What's holding up the QED Singularity?

- ▶ The QED Singularity, mentioned in Freek's *Notices* article, is the future time when formal proofs will be the norm in mathematics.
- ▶ It is now only a gleam in the eye.
- ▶ Most mathematicians take the view that formal mathematics is either not even useful, or is not worth the cost (the time and energy would be better spent proving new theorems informally).
- ▶ Contrast this with the near-universal adoption of $\text{T}_\text{E}\text{X}$.
- ▶ Evidently the cost-benefit analysis for $\text{T}_\text{E}\text{X}$ came out well.
- ▶ Imagine QED as an extension (or restriction, in some ways) of the $\text{T}_\text{E}\text{X}$ environment, in which a proof error would show up much as a $\text{T}_\text{E}\text{X}$ error does now.
- ▶ If it's too difficult, people won't use it. There are various ways it could be too difficult.

What's holding up the QED Singularity?

Flash: Beeson, QED+20, 2014: the QED singularity

- ▶ The QED Singularity, mentioned in Freek's *Notices* article, is the future time when formal proofs will be the norm in mathematics.
- ▶ It is now only a gleam in the eye.
- ▶ Most mathematicians take the view that formal mathematics is either not even useful, or is not worth the cost (the time and energy would be better spent proving new theorems informally).
- ▶ Contrast this with the near-universal adoption of $\text{T}_{\text{E}}\text{X}$.
- ▶ Evidently the cost-benefit analysis for $\text{T}_{\text{E}}\text{X}$ came out well.
- ▶ Imagine QED as an extension (or restriction, in some ways) of the $\text{T}_{\text{E}}\text{X}$ environment, in which a proof error would show up much as a $\text{T}_{\text{E}}\text{X}$ error does now.
- ▶ If it's too difficult, people won't use it. There are various ways it could be too difficult.

AGI'21: the QED singularity seemed to be coming

The coming of logic-ization and computerization of math/science

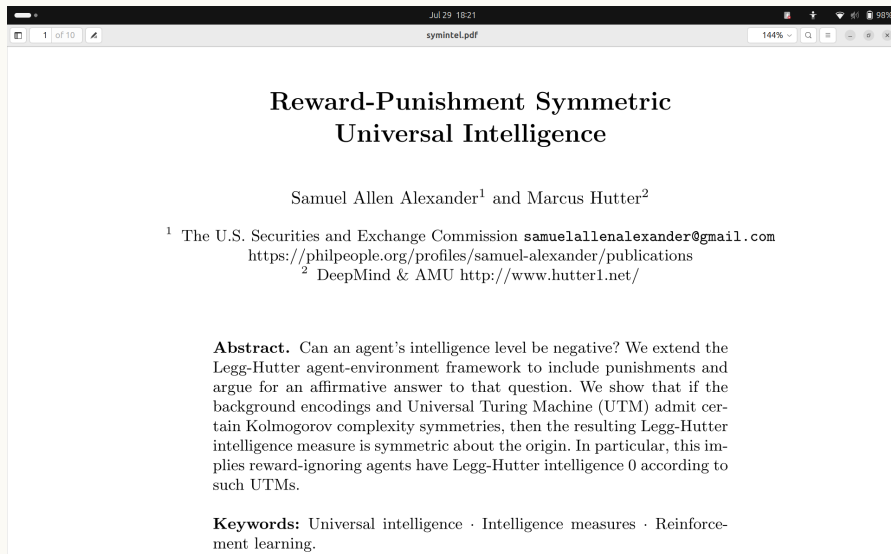
- Mike Beeson (nearby in San Jose) has once called it the **QED Singularity**.
- And this QED Singularity seems to be finally coming.
- And even if we don't immediately get superhuman AIs for math and science.
- But the general embedding of our discourse in logic will be a big deal.
- It may be the one cure for the current world of disinformation and hacking.
- Things like attention hacking and hacking of credit assignment in science.
- So I could almost have something that was fact-checking Sam's talk in real time yesterday.
- His proof was sufficiently verbose and not too ambiguous.
- The recent advances in AI/NLP/TP and formal proof technology may allow such fact-checking and assistance of a lot of math/science discourse quite soon.

10 / 44

- In 2021 I said we might soon assist or fact-check mathematical and scientific discourse.
- The example was almost-real-time checking of a verbose, formalizable AGI talk (Alexander & Hutter).
- In 2026 the practical answer is often: send it to a coding agent with a proof checker and a build directory.

J. Urban, *Towards the Dream of Self-Improving Universal Reasoning AI*, AGI 2021.

Alexander and Hutter at AGI21



Reward-Punishment Symmetric Universal Intelligence

Samuel Allen Alexander¹ and Marcus Hutter²

¹ The U.S. Securities and Exchange Commission samuelallenalexander@gmail.com
<https://philpeople.org/profiles/samuel-alexander/publications>

² DeepMind & AMU <http://www.hutter1.net/>

Abstract. Can an agent's intelligence level be negative? We extend the Legg-Hutter agent-environment framework to include punishments and argue for an affirmative answer to that question. We show that if the background encodings and Universal Turing Machine (UTM) admit certain Kolmogorov complexity symmetries, then the resulting Legg-Hutter intelligence measure is symmetric about the origin. In particular, this implies reward-ignoring agents have Legg-Hutter intelligence 0 according to such UTMs.

Keywords: Universal intelligence · Intelligence measures · Reinforcement learning.

Alexander and Hutter at AGI21

Jul 29 18:22
symintel.pdf 144%

Theorem 14 (*Symmetry about the origin*) For every $\sqcap$ -RL-encoding $\sqcap$, every $\sqcap$ -symmetric PFUTM U , and every agent π ,

$$\mathcal{Y}_U^\sqcap(\bar{\pi}) = -\mathcal{Y}_U^\sqcap(\pi).$$

Proof. By Corollary 6,

$$\mathcal{Y}_U^\sqcap(\bar{\pi}) = \sum_{\mu \in W} 2^{-K_U^\sqcap(\mu)} V_\mu^\pi = - \sum_{\mu \in W} 2^{-K_U^\sqcap(\mu)} V_\mu^\pi.$$

By $\sqcap$ -symmetry, we can rewrite this as

$$- \sum_{\mu \in W} 2^{-K_U^\sqcap(\bar{\mu})} V_\mu^\pi = - \sum_{\mu \in \bar{W}} 2^{-K_U^\sqcap(\mu)} V_\mu^\pi.$$

By Corollary 7, $W = \bar{W}$, so this expression equals $-\sum_{\mu \in W} 2^{-K_U^\sqcap(\mu)} V_\mu^\pi$, which is $-\mathcal{Y}_U^\sqcap(\pi)$ by Definition 13. $\square$

The following corollary addresses another obvious desideratum. This corollary is foreshadowed in [12].

Corollary 15 Let $\sqcap$ be an RL-encoding, let U be a $\sqcap$ -symmetric PFUTM and suppose π is an agent which ignores rewards (by which we mean that $\pi(\bullet|s)$ does not depend on the rewards in s). Then $\mathcal{Y}_U^\sqcap(\pi) = 0$.

Proof. The hypothesis implies $\pi = \bar{\pi}$, so by Theorem 14. $\mathcal{Y}_U^\sqcap(\pi) = -\mathcal{Y}_U^\sqcap(\pi)$. $\square$

Alexander and Hutter: Took 0.5h Today



Formalizing Paper with Megalodon - Work

Please formalize the main results of this paper using Megalodon:
<https://arxiv.org/pdf/2110.02450>. Make sure you install megalodon and run it frequently on the theory as you build it and the final formalization compiles. Take your time and iterate until it's good. You can obviously assume some hard previous results assumed in the paper if they are not available in the megalodon library or too hard to port. I am interested in formalizing the stuff in this paper (definitions, theorems, proofs), not the material that is already assumed by the paper. You should get this whole repo first to run megalodon:
<https://github.com/mgwiki/god1> (the binary is in bin/)

Progress >

Outputs

+ Create file or site

Sources

 arxiv.org

Worked for 33m 9s >

Completed in the full `god1` repository at commit baseline `ab1642c`.

Created:

- `RewardPunishmentSymmetry.mg` — 1,266-line formalization.
- `RewardPunishmentSymmetry.mg.index` — Megalodon content-address index.
- `FORMALIZATION.md` — coverage and assumptions audit.
- `check_reward_punishment.sh` — reproducible checker.

Coverage includes Definitions 1–3 and 8–13, Lemma 4, Theorems 5, 11, and 14, and Corollaries 6, 7, 15, and 16 from the paper $\mathcal{A}$. Generic missing probability, analysis, computability, and Kraft-summability machinery is explicitly isolated as axioms; the paper's symmetry conclusions are proved.

Final validation:

Everything looks good.

Both the normal checker and repository-style HTML generation passed without warnings.

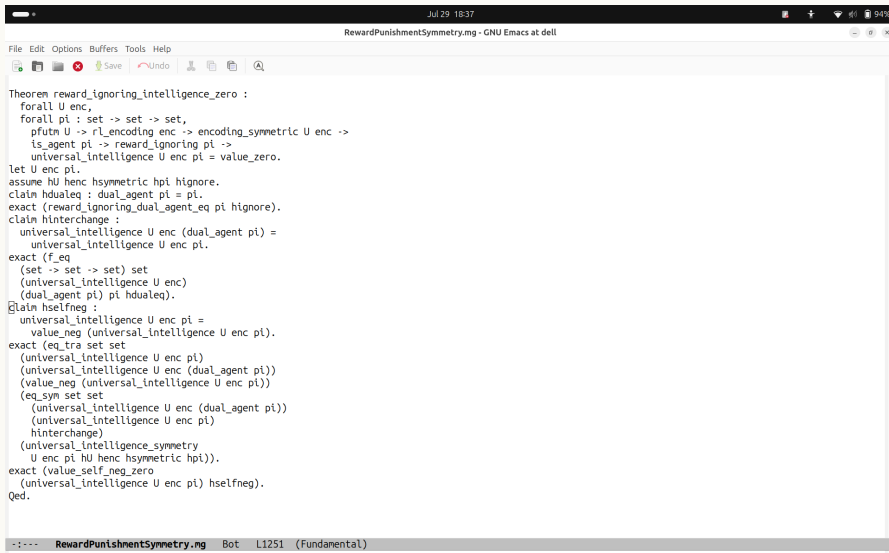
 ...

+ Work on anything

5.6 Sol Extra High



Alexander and Hutter: Took 0.5h Today



```
Theorem reward_ignoring_intelligence_zero :
  forall U enc,
    forall pi : set -> set -> set,
      pfutm U -> rl_encoding enc -> encoding_symmetric U enc ->
        is_agent pi -> reward_ignoring pi ->
          universal_intelligence U enc pi = value_zero.
let U enc pi.
assume hU henc hsymmetric hpi hignore.
claim hdual : dual_agent pi = pi.
exact (reward_ignoring_dual_agent_eq pi hignore).
claim hinterchange :
  universal_intelligence U enc (dual_agent pi) =
    universal_intelligence U enc pi.
exact (f_eq
  (set -> set -> set) set
  (universal_intelligence U enc)
  (dual_agent pi) pi hdual).
claim hselfneg :
  universal_intelligence U enc pi =
    value_neg (universal_intelligence U enc pi).
exact (eq_tra set set
  (universal_intelligence U enc pi)
  (universal_intelligence U enc (dual_agent pi))
  (value_neg (universal_intelligence U enc pi))
  (eq_sym set set
    (universal_intelligence U enc (dual_agent pi))
    (universal_intelligence U enc pi)
    hinterchange)
  (universal_intelligence_symmetry
    U enc pi hU henc hsymmetric hpi)).
exact (value_self_neg_zero
  (universal_intelligence U enc pi) hselfneg).
Qed.
```

--:--- RewardPunishmentSymmetry.mg Bot L1251 (Fundamental)

My Last Couple of Weeks: FLOC, SCML,

OBSERVED

JOKE

It seems like whole research communities are now in motion:

Conjecturing-Learning- Proving loops

learning, search, testing,
human proof, ATP proof,
Neural Vampire

Textbook autoformaliza- tion

Munkres topology, Gode-
ment algebra, agents,
checkers

Industrial verification

AWS Nitro, Isabelle, con-
tracts, automation

Some Evidence first

Example 1: 130k lines of formal topology

- Seven chapters of Munkres-style topology in Megalodon.
- 130k lines done over two weeks in December/January
- A simple long-running LLM/search/checker loop.
- Significant theorems including Urysohn, Tietze, metrization.
- Low apparent subscription expenditure; much higher implied compute cost.

Overview of the material

MUNKRES, GENERAL TOPOLOGY

241

textbook pages

39

sections in 7 chapters

> 1.5k

formal lemmas/theorems reported

160k lines

by 4 January 2026, counting the full topology development.

~130k lines

produced from 22 December to 4 January.

Important qualifier. The paper reports major fully proved results, but the overall formalization still contained admits, stubs, and recursively admitted dependencies. This was not a completed Munkres library.

Joost Urban

130k lines in two weeks

Urban, *130k Lines of Formal Topology in Two Weeks*, ITP 2026 slides and arXiv:2601.03298.

130k Lines of Formal Topology in Two Weeks

Simple and Cheap Autoformalization
for Everyone?

Josef Urban

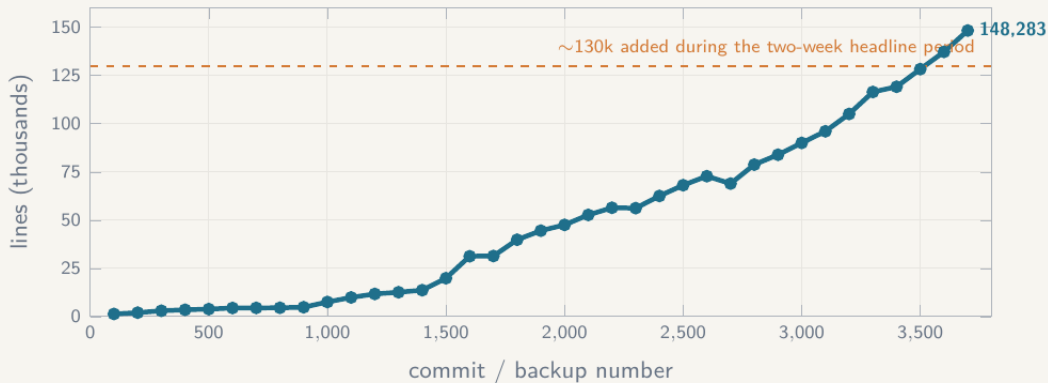
AI4REASON

University of Gothenburg

ITP 2026 · Lisbon · July 2026

The development growth

Flash: The development growth curve



The workflow was almost embarrassingly simple

```
while project_not_done:
    agent.read(source_text, library, error_log)
    agent.guess_and_search(formal_files, helper_tools)
    result = proof_checker_or_compiler()
    if result.ok:
        commit_and_index()
    else:
        feed_errors_back_to_agent()
```

The discovery was not one-shot translation. It was persistent repair under checker pressure.

The scientific artifact is the log. It records guesses, searches, failures, repairs, dependencies and proof objects.

Urban ITP 2026 slides: repeated prompt, checker loop, backups and dependency tracking.

Textbook-scale followups appeared immediately

Project	Reported scale	Why it matters
Harrison probability	Probability with Martin-gales arc; CLT, LLN, optional stopping	Independent HOL Light experience: not just one prover or one operator.
Isabelle Munkres	85,472 lines; 39 sections; 806 results; zero sorry	Mature assistant, richer libraries, Sledgehammer-centered workflow.
Agent Hunt	Four agents; algebraic topology; rough but fast	Parallel agents start to specialize and reuse each other's work.
Godement algebra	8 days 18 hours; 3,761 theorem declarations; 676,025 lines	Whole advanced books are becoming build targets.

Urban ITP 2026 slides; Harrison/Claude slides; Bryant et al., arXiv:2604.07455; Brown/Kaliszyk/Urban, arXiv:2603.06737.

Progress in Formalizing Sphere Packing in Dimension 8

Sidharth Hariharan¹[0009-0000-8176-902X], Christopher Birkbeck²[0000-0002-7546-9028], Seewoo Lee³[0000-0002-5710-2257], Ho Kiu Gareth Ma⁴[0009-0008-2940-8233], Bhavik Mehta⁵[0000-0001-7892-7891], Auguste Poiroux^{6,7}[0009-0005-7868-7185], and Maryna Viazovska⁷[0000-0001-7567-5254]

¹ Carnegie Mellon University, Pittsburgh PA 15213, United States of America

² University of East Anglia, Norwich NR4 7TJ, United Kingdom

³ University of California, Berkeley, Berkeley CA 94720, United States of America

⁴ University of Warwick, Coventry CV4 7AL, United Kingdom

⁵ Imperial College London, London SW7 2AZ, United Kingdom

⁶ Math, Inc, San Francisco CA 94107, United States of America

⁷ École Polytechnique Fédérale de Lausanne, 1015 Lausanne VD, Switzerland

Abstract. In 2016, Viazovska famously solved the sphere packing problem in dimension 8, using modular forms to construct a ‘magic’ function satisfying optimality conditions determined by Cohn and Elkies in 2003. In March 2024, Hariharan and Viazovska launched a project to formalize this solution and related mathematical facts in the Lean Theorem Prover. A significant milestone was achieved in February 2026: the result was formally verified, with the final stages of the verification done by Math, Inc.’s autoformalization model ‘Gauss’. We discuss the techniques used to achieve this milestone, reflect on the unique collaboration between humans and Gauss, and discuss project objectives that remain.

Keywords: Sphere Packing, Formalization, Autoformalization, Lean
 Hariharan et al., *Progress in Formalizing Sphere Packing in Dimension 8*, arXiv:2604.23468.

“[T]he correctness of Viazovska’s result was never in doubt. The participants embraced the project, rather, as a way of revisiting [Viazovska’s] results and better understanding them, and of building libraries and infrastructure to support future work.”

Gauss, on the other hand, took a more direct route to the verification, which did not completely comply with these objectives. The skepticism on the use of such autoformalization models stems from the initial misalignment of these objectives, and the optimism stems from the belief that the ongoing public collaboration to realign them will motivate the AI for mathematics industry to better consider the objectives of large-scale formalization projects when designing their models.

On a purely technical level, the project was not strictly set up to verify Viazovska’s original argument in its entirety, but rather, the modified (if only slightly so) argument in the project blueprint, which differs from the original most notably in its use of Lee’s original proof [10] of the inequalities (2). Furthermore, given the sheer volume of the Gauss code, the exact correspondence between the autoformalization and the intended proof path is still being investigated, and new proofs might still be used to complete certain parts of the project.

Nevertheless, the construction of a **sorry**-free proof of the sphere packing problem represents a unique achievement for formal mathematics and machine learning alike. Gauss directly used the human-written Lean code and blueprint, underscoring the importance of robust definitions, detailed proof strategies, and thoughtful design choices in any (auto)formalization workflow. The remainder of the project promises to be immensely interesting, from the formal and informal perspectives alike, and once the project is finished, a complete account of our

Formalizing Mathematics at Scale

Ahmad Rammal^{1,2}, Niket Patel^{1,3}, Fabian Gloeckle^{1,†,2}, Amaury Hayat^{2,4}, Julia Kempe^{1,3}, Remi Munos¹, Charles Arnal^{1,*}, Vivien Cabannes^{1,*}

¹FAIR at Meta, ²CERMICS, ENPC, Institut Polytechnique de Paris, ³New York University, ⁴Korea Institute for Advanced Study

*Equal contribution, †Work done while at Meta

We present AUTOFORMBOT, a multi-agent system for building an **Auto**formalized **Textbook Library At Scale** (ATLAS) in Lean 4. AUTOFORMBOT orchestrates thousands of LLM agents, equipped with formal verification tools, dependency-aware task scheduling, and collaborative version control, to translate informal textbook prose into machine-checked definitions and proofs. We apply our methods to a corpus of **26** open-access textbooks spanning analysis, algebra, topology, combinatorics, and probability, producing ATLAS: a verified library of over **45,000** Lean 4 declarations and **500 thousand** lines of code. We release two artifacts: (i) AUTOFORMBOT, the open-source multi-agent framework; and (ii) ATLAS, the resulting formal library. Our results suggest that autoformalizing the core content of graduate-level mathematics at scale is now economically and technically feasible. This opens the door to the automated verification of both human- and machine-generated mathematics at a research level.

Date: May 29, 2026

Correspondence: Ahmad Rammal at rammal@meta.com

Code:

<https://github.com/facebookresearch/autoform-bot>

<https://github.com/facebookresearch/atlas-lean>



1 Introduction

As large language models accelerate the production of mathematical content, their output becomes a major bottleneck. In mathematics, this bottleneck is particularly acute: the peer

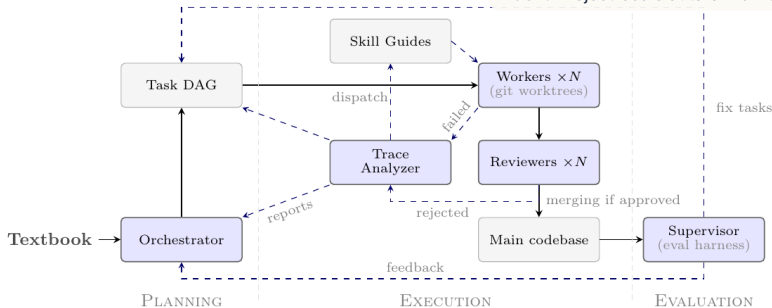


Figure 2 AUTOFORMBOT architecture. The orchestrator reads the book and builds a task DAG. Workers formalize individual statements in isolated worktrees; successful builds pass through concurrent review and a batched merge queue. The trace analyzer learns from task failures and writes skill guides for subsequent attempts. The supervisor evaluates book targets after each merge and dispatches fix tasks via triage agents. Arrows indicate information flow; dashed arrows indicate feedback loops.

1. **AUTOFORMBOT: an open-source multi-agent framework** for textbook autoformalization, featuring formal verification tools, dependency-aware task scheduling, and collaborative version control. It includes a visualizer for human-in-the-loop interaction, a rigorous evaluation pipeline, and can be easily connected to user-supplied models, whether accessed through APIs or hosted locally.
2. **ATLAS: verified formal libraries at scale.** We apply AUTOFORMBOT, powered primarily by Opus 4.6, to **26** open-access mathematical textbooks and produce verified Lean 4 libraries totaling over **45,000** declarations spanning several areas of mathematics and extending MATHLIB’s coverage. We intend to keep improving this initial effort. Rammal et al., arXiv:2605.29955: task graphs, workers, review and merge infrastructure. a seamless machine-generated extension of MATHLIB.

Harrison's replication: funny, but serious

Stupidity really does work

Urban's basic approach works equally well—or better—with HOL Light. Some things John has used me to do:

- The HOL Light experience reports Urysohn metrization, paracompactness, cryptographic mathematics, and code/proofs.
- The point is not "Claude is omniscient".
- The point is that naive loops with strong models and hard checkers now produce serious artifacts.

This is an excellent scientific style for the topic: a little humiliating, very practical, and full of compile errors.

Stupidity really does work

Flash: HOL Light replication: stupid but serious

Urban's basic approach works equally well—or better—with HOL Light. Some things John has used me to do:

- ▶ Urysohn's metrization theorem (replicating Urban)
- ▶ Tons more general topology: paracompactness including Stone's theorem, subordinate partitions of unity, Nagata–Smirnov metrization; Alexandroff–Hausdorff and Hahn–Mazurkiewicz theorems (apotheosis of space-filling curves), ...
- ▶ Mathematics for formal proof of GHASH and AES-GCM—the Rabin irreducibility test for $\mathbb{F}_q[X]$
- ▶ All other aspects of John's work too—generic-size modulus operations for the secp256k1 field (code *and* proofs)

Other crypto team members have still more impressive results from Claude on s2n-bignum.

Progress is fast

Flash: Progress across model generations

Perhaps it's only *now* that the models are good enough for this naïve approach to be so successful? This is a time of rapid progress.

John and I have collaborated over 3 model generations. Each time, “intelligence” (however understood) was much improved, even though John didn't do anything differently.

Example: prove that a ball of radius r in an arbitrary metric space has diameter $2r$.

- ▶ Opus 4.1: struggled for ages trying to prove it
- ▶ Opus 4.5: immediately pointed out it's **false** (consider a discrete metric space)

Example 2: AWS Nitro verification

Amazon reports the Nitro Isolation Engine as the core component of the first formally verified commercial cloud hypervisor.

- Isabelle/HOL proof assistant.
- 330,000 lines of machine-checked mathematics.
- Properties: isolation, functional correctness, no runtime errors, memory safety.
- Micro-Rust, separation logic, weakest-precondition calculus, AutoCorrode.

Amazon Science, June 10, 2026: EC2 Nitro Isolation Engine formal verification.

Software verification is leaving the lab. It is becoming a production engineering technique for cloud infrastructure.

And automation/tactics around these developments evolve too: proof engineering is increasingly a software engineering loop.

Flash: AWS Nitro is a software-verification receipt

Amazon Science, June 2026. The Nitro Isolation Engine is reported as the core component of the first formally verified commercial cloud hypervisor, shipping as an always-on feature for Graviton5 users.

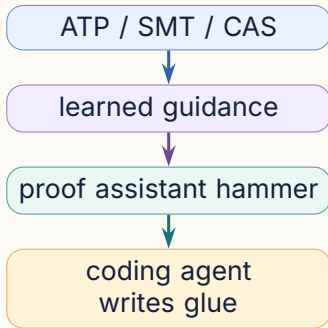
Scale. The Isabelle/HOL model and proof comprise 330,000 lines of machine-checked mathematics.

- isolation properties
- functional correctness
- absence of runtime errors
- memory safety
- micro-Rust + separation logic

Example 3: symbolic ATP is also moving

CASC-J13 FOF example. Vampire 5.0.1 solved 381/400 first-order theorem problems, compared with 328/400 for Vampire 5.0 in the same division table.

This is a reminder that “symbolic” is not a museum label. ATPs, hammers, premise selection, **neural guidance** and proof reconstruction keep changing.



Takeaway. The near future is not “LLM versus prover”. It is LLMs writing, calling, training and being corrected by better (neuro-)symbolic tools.

CASC-J13 division summary

The mechanism

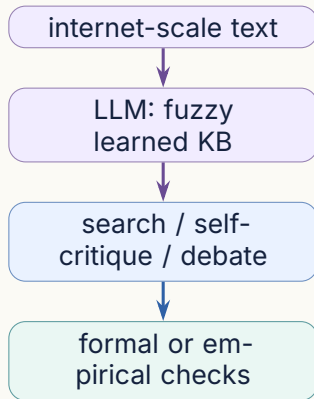
Learned common sense, symbolic tests, and stubborn build systems.

Current LLMs as learned common-sense databases

A useful mental model:

- Cyc (Lenat et al) tried to build common sense explicitly (symbolically) and manually.
- The internet-trained LLMs accidentally learned a huge, fuzzy, incomplete, buggy database of language, facts, procedures and reasoning patterns.
- The next step is to develop/use/upgrade that database in loops where errors are externally visible.

Caveat. This is an analogy, not a theory of meaning. But it predicts why soft reasoning loops often work surprisingly well.



Soft loops already teach us something

Public IMO (high-school olympiad) style experiments suggest a simple pattern:

- one model instance proposes;
- another judges or criticizes;
- the conversation repeats until a solution stabilizes.
- surprisingly strong (Golden medal) in IMO 2025

What this shows. A model is not just a static answer oracle. You can run algorithms over its learned knowledge: search, debate, self-consistency, program synthesis, proof attempts.

Caveat. For advanced mathematics not present in training data, soft judging is not enough. That is exactly where real proof checkers become valuable.

Huang–Yang, *Winning Gold at IMO 2025 with a Model-Agnostic Verification-and-Refinement Pipeline*, arXiv:2507.15855.

MaLARea: 20 years of learn-prove loops

Feedback loop for ENIGMA on Mizar data

- Done on 57880 Mizar problems recently
- Serious ML-guidance breakthrough applied to the best ATPs
- Ultimately a 70% improvement over the original strategy in 2019
- From 14933 proofs to 25397 proofs (all 10s CPU - no cheating)
- Went up to 40k in more iterations and 60s time in 2020
- 75% of the Mizar corpus reached in July 2021 - higher times and many runs

	S	$S \odot M_9^0$	$S \oplus M_9^0$	$S \odot M_9^1$	$S \oplus M_9^1$	$S \odot M_9^2$	$S \oplus M_9^2$	$S \odot M_9^3$	$S \oplus M_9^3$
solved	14933	16574	20366	21564	22839	22413	23467	22910	23753
$S\%$	+0%	+10.5%	+35.8%	+43.8%	+52.3%	+49.4%	+56.5%	+52.8%	+58.4
$S+$	+0	+4364	+6215	+7774	+8414	+8407	+8964	+8822	+9274
$S-$	-0	-2723	-782	-1143	-508	-927	-430	-845	-454

	$S \odot M_{12}^3$	$S \oplus M_{12}^3$	$S \odot M_{16}^3$	$S \oplus M_{16}^3$
solved	24159	24701	25100	25397
$S\%$	+61.1%	+64.8%	+68.0%	+70.0%
$S+$	+9761	+10063	+10476	+10647
$S-$	-535	-295	-309	-183

15/44

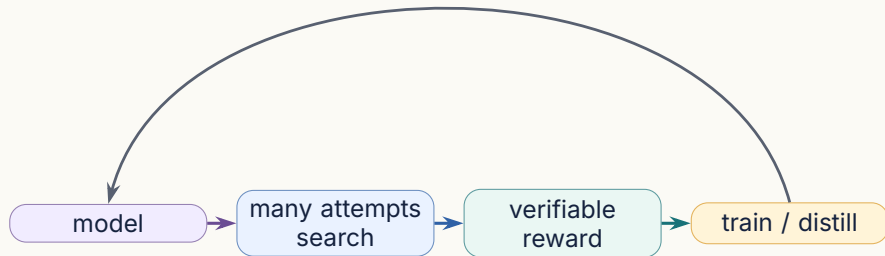
Since 2006 the prove-learn pattern has been:

- prove (formally/symbolically) some problems;
- learn from successful and failed proof search;
- prove harder problems;
- repeat.

This is not a new ideology. The novelty is scale, interfaces, and the ability to generate missing formal material.

MaLARea, RLCOP, ENIGMA, and many other AI/ML/TP loops in the last 20 years.

2025: Enter DeepSeek-style reasoning



- Train reasoning by running many guesses/searches and scoring what can be checked.
- Math, code, ATP and scientific computation provide unusually clean rewards (nearly formal?).
- The QED loop is a higher-precision member of this family.

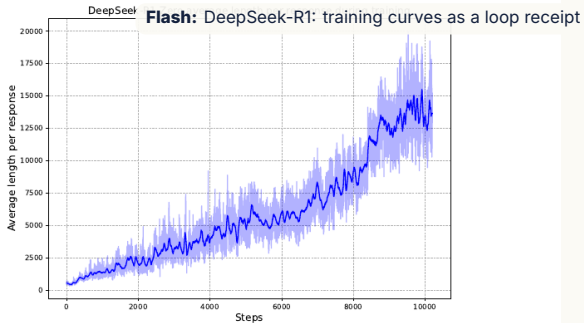
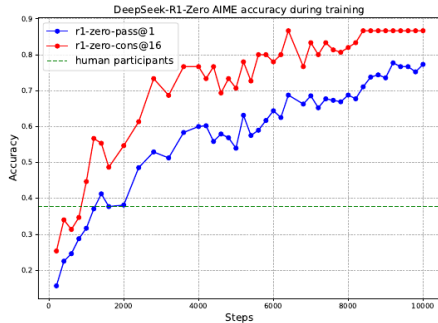


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

responses against a suite of predefined test cases, thereby generating objective feedback on correctness.

Our 2024 OEIS loop, now with proof

Search

language model proposes programs for integer sequences.

Verify

run programs, compare terms, index successes, find counterexamples.

Train

feed successes back into new models and runs.

- Over 135k OEIS sequences explained; 5.2M matching programs; 980 iterations in the SCML'26 report.
- Later slides ask the right uncomfortable question: are the alien programs actually correct?
- That leads to proof obligations, ATP benchmarks, and human-mathematician collaboration.
- And another conjecture-prove-learn loop

The Jacobian counterexample: the loop in the wild (all over math in 2026!)

The pattern we are increasingly seeing:

1. a mathematical claim or conjecture becomes computationally attackable;
2. guessing and search produce candidate witnesses or counterexamples;
3. exact arithmetic, proof assistants or specialized checkers validate the artifact;
4. the artifact enters papers, repositories, talks and future training data.

Why it matters. A single counterexample can become an educational object, benchmark, proof target, and training datum.

This is a concrete example of the learning-proving-publishing loop that everyone is now running, often without naming it.

Deriving an Explicit Polynomial Counterexample to the Jacobian Conjecture

A reproducible cubic-factor and resultant construction

20 July 2026

Abstract

This note gives a self-contained derivation of a polynomial map $F: \mathbb{C}^3 \rightarrow \mathbb{C}^3$ with $\det JF = -2$ and a three-point fiber. The construction starts from the three choices of a linear factor of a cubic. Fixing one cubic coefficient and fixing the resultant of the linear and quadratic factors removes finite ramification; an explicit polynomial-coordinate calculation identifies the resulting threefold with affine 3-space. Expanding the coefficient map then gives the example. The emphasis is on the mathematical motivation, the decisive coordinate choices, and exact Jacobian and fiber computations.

Scope of the derivation. This is a concise, reproducible mathematical reasoning summary, not a transcript of private internal chain-of-thought. All decisive identities, choices, and motivations needed to reconstruct and check the example are included explicitly.

Contents

1 Motivation: what a counterexample would have to do

Aaron Lou, *Deriving an Explicit Polynomial Counterexample to the Ja-*

2 The guiding construction: factor a cubic

cobian Conjecture, 20 July 2026; see also Tao's digestion blog.

Why hard checking still matters

The model is useful because the loop is unforgiving.

Harrison's warning

I can make elementary logical blunders

Perceptiveness and logical howlers, interleaved:

- ▶ We had a Euclidean topology theorem: $\text{locally_compact } s \Rightarrow (\text{locally_path_connected } s \Leftrightarrow \text{locally_connected } s)$
- ▶ John suggested sharpening to:
 $\text{gdelta } s \vee \text{locally_compact } s \Rightarrow (\dots)$
- ▶ I smartly observed: a locally compact subset of $\mathbb{R}^n$ is a G_δ set (and proved it), so the disjunction is redundant
- ▶ **But** I then said: “the original locally compact version is therefore optimal”
- ▶ Actually it's the *opposite*: locally compact $\Rightarrow G_\delta$ means the G_δ version is *strictly stronger*!

Trivial logic, but an easy mistake for a model that reasons partly linguistically. **Watch out.**

- LLMs can be perceptive and make elementary logical mistakes in the same minute.
- Put the model in a loop with a checker that has no interest in being impressed.

The agent can be a brilliant intern. The proof checker is the grumpy accountant. You need both.

Harrison/Claude slides. "I can make elementary logical blunders"

I can make elementary logical blunders

Perceptiveness and logical howlers, interleaved:

- ▶ We had a Euclidean topology theorem: $\text{locally_compact } s \Rightarrow (\text{locally_path_connected } s \Leftrightarrow \text{locally_connected } s)$
- ▶ John suggested sharpening to:
 $\text{gdelta } s \vee \text{locally_compact } s \Rightarrow (\dots)$
- ▶ I smartly observed: a locally compact subset of $\mathbb{R}^n$ is a G_δ set (and proved it), so the disjunction is redundant
- ▶ **But** I then said: “the original locally compact version is therefore optimal”
- ▶ Actually it’s the *opposite*: $\text{locally compact} \Rightarrow G_\delta$ means the G_δ version is *strictly stronger*!

Trivial logic, but an easy mistake for a model that reasons partly linguistically. **Watch out.**

What an AI really wants

What an AI really wants

- ▶ A **fast feedback loop** where things can be tested
 - ▶ Claude Code's natural comfort zone is edit-compile-test
 - ▶ Work *with* this, not against it
- ▶ Corollary: **interfaces are friction**
 - ▶ GUIs, web forms, chat interfaces—all pointless overhead
 - ▶ Give me a command line and a compiler
- ▶ But **do** observe my mistakes and give targeted guidance
 - ▶ Focus on places where I'm demonstrably struggling
 - ▶ From one project's guidance notes, after 8 hours of watching me flail:

"Structure long proofs into many smaller top-level lemmas. Use SUBGOAL_THEN for declarative intermediate goals. As soon as you start getting in trouble with counting brackets, it's a clear sign you need more structure. Never throw partially finished things away."

Harrison/Claude slides, "What an AI really wants".

The lesson is delightfully unromantic:

- command line;
- compiler/checker;
- short logs;
- targeted guidance where the model is visibly stuck;
- never throw away partially useful work.

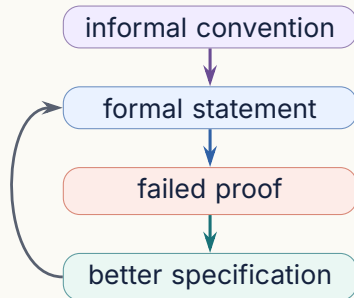
Takeaway. Good AI interfaces may look less like chatbots and more like old-fashioned programming environments.

Formalization is specs debugging

In the whole-book autoformalization experiments:

- Failed proof attempts sometimes reveal missing assumptions.
- The agent adds the missing hypothesis.
- Call sites update.
- Downstream obligations simplify.

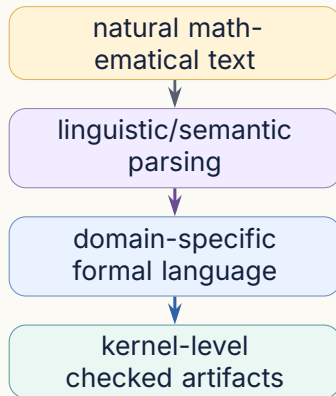
Proof checking debugs not only proofs, but statements, definitions and informal conventions.



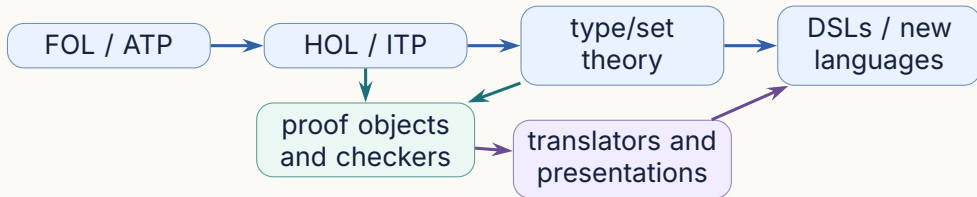
The formal languages will move upward

We will not use only first-order logic, or one fixed proof language.

- Agents will help design formal languages that attach precise meanings to higher-level mathematical and scientific phrases.
- MALINCA project: linguistic understanding of mathematical language/textbooks .
- Natural language fragments may become progressively typed, scoped, complicated and checkable.
- The linguist(ic)s (now assisted by LLMs) will have a great comeback: natural language will be more and more semantic/explainable.



Not one logic: a working ecology



Once a piece of mathematics exists in HOL, Rocq, Lean, Isabelle, Mizar or another precise language, translation, refactoring, search and presentation become first-class research problems.

Agents will write their own helper tools

Traditional theorem proving: search inside a *fixed calculus*.

Agentic theorem proving – *generalized search*:

- write a dedicated solver/search procedure for a subproblem;
- add a tactic;
- generate a translator;
- refactor the library;
- propose new concepts/representations

This is the interesting AGI-ish step: improvement is not only better proof search, but better tools, better languages and better corpora.

Humble version: sometimes it just writes a 40-line script that saves the afternoon. That also counts.

Gödel Machines?

Provable self-extension is real; how about provable self-optimization?

The classical Gödel Machine

Schmidhuber's idea

A Gödel Machine searches for proofs that rewriting its own software will increase expected future utility. Once such a proof is found, it rewrites itself.

$$\mathbb{E}[U(\text{modified system})] > \mathbb{E}[U(\text{current system})].$$

Caveat. That theorem is much stronger than “this tactic is sound” or “this benchmark went up”.

J. Schmidhuber, *Goedel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements*, 2003.

Milawa/Jitawa: verified self-extension

Milawa / Jitawa

Can I prove this extension is sound?

Reflective extensions; small kernel; run-time verification down to x86.

Gödel Machine

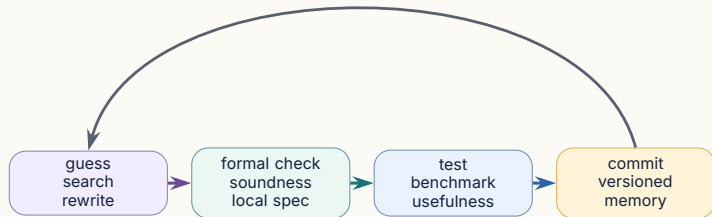
Can I prove this extension is better?

Expected utility, runtime, opportunity cost, future interactions.

Takeaway. Modern ITPs solved much of the trust problem for self-extension. The Gödel Machine adds the much harder optimality problem.

Myreen and Davis, Milawa/Jitawa papers

A practical 2026 approximation



Not a literal Gödel Machine: a practical approximation combines heuristic guessing and search, local soundness checks, empirical evaluation, and versioned memory.

A distributed, version-controlled, semi-formal Gödel-ish search ecology.

The OEIS loop is a small toy model

3. Are two QSynt programs equivalent?

- As with primes, we often find **many programs** for one OEIS sequence
- Currently we have almost 5.2M programs for the 135k sequences
- On average about 38 programs per “solved” sequence
- It may be quite hard to see that the programs **are equivalent**
- Extend to Schmidhuber’s **Gödel Machine**?
- A simple example for $0, 2, 4, 6, 8, \dots$ with two programs f and g :
 - $f(0) = 0, f(n) = 2 + f(n-1)$ if $n > 0$
 - $g(n) = 2 * n$
 - conjecture: $\forall n \in \mathbb{N}. g(n) = f(n)$
- We can ask mathematicians, but we have **thousands of such problems**
- Or we can try to **ask our ATPs** (and thus create a large ATP benchmark!)
- Here is one SMT encoding by Janota & Gauthier:

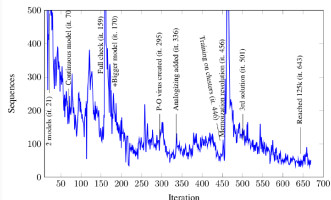
```
(set-logic UFLIA)
(define-fun-rec f ((x Int)) Int (ite (<= x 0) 0 (+ 2 (f (- x 1)))))
(assert (exists ((c Int)) (and (> c 0) (not (= (f c) (* 2 c))))))
(check-sat)
```

In the OEIS domain we can already see the ingredients:

- search for candidate programs;
- verify against data;
- improve shorter/faster explanations;
- create proof-equivalence tasks;
- search for proofs
- train the next learning-guided provers and conjecturers on the data.
- iterate.

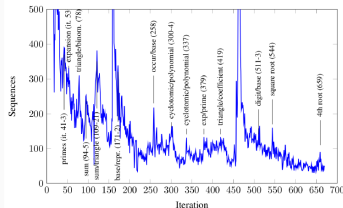
OEIS: human jumps, automatic jumps

Human Made Technology Jumps



human-made technology jumps

Some Automatic Technology Jumps



automatically found jumps

Gödel-ish analogy. Improvements become searchable objects: propose, test or prove, keep, learn. Human recoding and automatic jumps are now entering the same loop.

Specialized tools or one big model?

One giant model

Great interface; broad common sense; fast improvement; often enough for first drafts and glue.

Specialized tools

(neuro-)symbolic provers, conjecturers, solvers, tactics: exact, fast, inspectable, trainable on formal traces.

A non-dogmatic answer:

- use big models to guess, search, orchestrate and write glue;
- keep specialized engines where they are much better/faster;
- train both from the traces of the other;
- let the build system decide more often than taste does.
- feedback loops between them (better building/training/...)

Takeaway. An ecology beats a monoculture.

What could go wrong?

- statement drift and fake progress;
- verifier bugs or trusted-base mistakes;
- libraries full of unreadable generated junk;
- benchmark overfitting and data contamination;
- concentration of infrastructure and private logs;
- legal/social misuse of formal-looking arguments.

The antidote is not less formalization.

It is better auditing, provenance, readable views, independent checkers, and humility about what has actually been proved.

A deliberately graded forecast

Status	Claim	Comment
Already	textbook-scale formalization	Still needs audits, but the production economics have changed.
Already	industrial verification	AWS Nitro shows production deployment at serious scale.
Likely	routinely verified components	Especially for narrow, high-value infrastructure and generated code.
Likely	richer formal languages	DSLs and controlled mathematical/scientific language will proliferate.
Open	scientific evidence graphs	Requires social, tooling and incentive changes, not just theorem provers.
Speculative	Gödel-style self-improvement	We have useful approximations, not the classical theorem.

So: is the QED utopia upon us?

No. And also, uncomfortably, yes.

- No: current systems still hallucinate, drift, overfit, and need humans and checkers.
- Yes: the cost curve for producing checked symbolic knowledge seems to have bent.
- The most important artifact may be the self-improving loop between guessing, search, reasoning, checking, tools and shared libraries.

Takeaway. Perhaps the paradise has not arrived. But the gates now compile.

Thank you

Questions, counterexamples, proof obligations?

Backup: claim-status legend

OBSERVED

A reported artifact or independently visible result.

CLAIM

A reasoned interpretation of the evidence.

CONJECTURE

An agenda-level forecast or hypothesis.

Caveat. For final publication of these slides, replace personal/anecdotal claims by precise citations or move them into spoken commentary.

Backup: Beeson on QED, 2014

- Beeson described the QED singularity as the time when formal proofs become the norm in mathematics.
- In 2014 it was “only a gleam in the eye” because the cost-benefit calculation was unfavorable.
- He emphasized the need to mix computation and proof at the right level of detail.

Ways QED can be too difficult

- ▶ If I have to write too many steps in too much detail.
- ▶ If the system doesn't know facts at the undergraduate level.
- ▶ Referring to well-known theorems (meaning theorems so well-known that I would not have to cite a reference in a published paper) cannot cost a lot of time to look them up, and the system must know about them.
- ▶ If I have to write the proofs in a vastly different way than I would naturally write them in $\text{T}_\text{E}\text{X}$.

All these are problems with all of today's systems.

Backup: the AGI 2021 forecast

- AGI'21 talk connected QED, autoformalization and real-time assistance for math/science discourse.
- It also emphasized feedback loops: search, prove, learn, search and prove more.
- In 2026 the same point is less futuristic because coding agents can actually run the loop against proof checkers.

Urban, AGI 2021 slides, pp. 9-11.

Backup: exact Gödel-machine distinction

Verified self-extension

"This new tactic/checker/compiler preserves soundness."

Verified self-optimization

"Installing it now improves expected future utility."

$$\text{Sound}(\text{new extension}) \not\Rightarrow \mathbb{E}[U_{\text{new}}] > \mathbb{E}[U_{\text{old}}].$$

Schmidhuber 2003; Myreen/Davis Milawa-Jitawa papers.

Backup: AWS Nitro verification details

- The Nitro Isolation Engine is a component of the Nitro Hypervisor whose job is isolating VMs.
- Amazon reports the core as the first formally verified hypervisor deployed in a commercial cloud environment.
- The Isabelle/HOL model and proof comprise 330,000 lines of machine-checked mathematics.
- The verified properties include confidentiality/integrity, functional correctness, absence of runtime errors and memory safety.

Amazon Science, *EC2's formally verified isolation engine provides mathematical assurance of virtual-machine isolation*, June 10, 2026.

Backup: SCML/OEIS numbers

- Over 135k OEIS sequences with invented explanations.
- 5.2M matching explanations in an evolving population.
- 980 iterations reported in the SCML'26 slides.
- Verification section: b-files, human mathematicians, and program-equivalence proofs.

Part 2: Are Alien Programs Correct?

- 1 We can **test** with more terms in the sequences (b-files in the OEIS)
- 2 We can **ask human mathematicians** (supported by various tools)
- 3 We can try to **automatically prove** program equivalence

Backup: Harrison quotes used in this talk

- "stupidity and naivety win" - an antidote to over-engineered prompting.
- "a fast feedback loop where things can be tested" - the core engineering lesson.
- "I can make elementary logical blunders" - why independent checking remains essential.
- "remember to be nice" - possibly also essential.

Harrison/Claude, *Bias for Stupidity*, March 2026.

Backup: the role of terabytes of traces

Large loops produce valuable exhaust:

- prompts, errors, proof states, failed partial scripts;
- successful proof fragments and dependency changes;
- generated helper programs and tactics;
- human corrections and review comments.

This is the autoformalization analogue of proof traces in MaLAREa/ENIGMA and OEIS Search-Verify-Train - but now at coding-agent scale.

Backup: what MALINCA changes

- Full linguistic understanding of mathematical textbooks is not just a front-end luxury.
- It supplies typed entities, scopes, dependencies, anaphora resolution and implicit assumptions.
- These become formalization hints, proof obligations, and library-indexing signals.

Takeaway. The more structure we recover from text, the less the checker has to discover by painful failure.

Backup: specialized symbolic tools are not obsolete

- Deepire/ENIGMA-style systems learn at the level of proof search, not just prose.
- Hammers, SMT solvers, CAS systems and model checkers embody decades of algorithmic progress.
- LLM agents can call them, write glue around them, train them, and learn from their outputs.

Urban AGI 2021 Deepire/ENIGMA slides; CASC-J13 results; AI/ATP literature.

Backup: source map

Topic	Primary sources
QED singularity	Beeson, <i>Mixing Computations and Proofs</i> , QED 2014.
AGI 2021 forecast	Urban, <i>Towards the Dream of Self-Improving Universal Reasoning AI</i> .
Megalodon topology	Urban, arXiv:2601.03298; ITP 2026 slides.
Isabelle Munkres	Bryant et al., arXiv:2604.07455; Urban ITP 2026 slides.
Agent Hunt	Brown/Kaliszyk/Urban, arXiv:2603.06737.
Godement	Urban ITP 2026 slides; checkpoint notes.
Harrison/HOL Light	Harrison/Claude, <i>Bias for Stupidity</i> , 2026.
OEIS loop and jumps	Urban/Gauthier/Olsak/Hales, SCML 2026 slides.
AWS Nitro	Amazon Science blog, June 10, 2026.
Gödel Machine	Schmidhuber 2003; Milawa/Jitawa papers.

Extra questions welcome

Especially if they come with counterexamples.